S.No: 4	Exp. Name: ***Minimum Spanning Tree using Kruskal's Algorithm***	Date:2026-08-04

ID: 25070521300

Aim:

Write a C program to construct the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Kruskal's Algorithm.

You are provided with a driver code. Complete the function **kruskalMST()** so that the program correctly finds and prints the Minimum Spanning Tree and its total cost. The graph is represented using an adjacency matrix.

Implement the function:

```
void kruskalMST(int **cost, int V)
```

Where,

- $V \rightarrow$ Number of vertices in the graph
- $\text{cost} \rightarrow$ Adjacency matrix storing edge weights

Input Format:

- The first line of input reads an integer V by prompting "No of vertices: ", representing the number of vertices in the graph.
- The next $V \times V$ integers represent the adjacency matrix of the graph with a heading "Adjacency matrix:"
- $\text{cost}[\text{i}][\text{j}]$ indicates the weight of the edge between vertex i and vertex j
- If there is no edge between two vertices, the weight should be given as 9999 (representing infinity).
- The graph is undirected, so the matrix is symmetric

Output Format:

1.The output should display each edge selected for the Minimum Spanning Tree in the order they are chosen in the below format:

```
Edge k:(u, v) cost:w
```

Where:

- k is the edge number starting from 0
- u and v are the vertices connected by the edge
- w is the weight of the edge

2.In the last line the program should print the total minimum cost of the MST as follows:

```
Minimum cost= X
```

Where X is the total cost of the Minimum Spanning Tree.

Constraints:

- $1 \leq V \leq 100$
- Edge weights are non-negative integers
- The adjacency matrix is dynamically allocated
- Exactly $V - 1$ edges must be included in the MST

Note:

- Process the edges in ascending order of their weights. For edges with the same weight, follow their row-wise order in the upper triangular part of the adjacency matrix.
- Refer to the visible test cases to strictly match the input and output layout.

2025-29-CSE-E

Symbiosis Institute of Technology | SIT Nagpur

Source Code:

minCostFinding.c

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>


// Structure to represent an edge
typedef struct {
    int u;
    int v;
    int weight;
} Edge;

// Structure to represent a subset for Union-Find
typedef struct {
    int parent;
    int rank;
} Subset;

// Find set of an element i (uses path compression)
int find(Subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

// A function that does union of two sets of u and v (uses union by rank)
void Union(Subset subsets[], int u, int v) {
    int root_u = find(subsets, u);
    int root_v = find(subsets, v);

    if (subsets[root_u].rank < subsets[root_v].rank)
        subsets[root_u].parent = root_v;
    else if (subsets[root_u].rank > subsets[root_v].rank)
        subsets[root_v].parent = root_u;
    else {
        subsets[root_v].parent = root_u;
        subsets[root_u].rank++;
    }
}

// Comparison function for qsort.
// Maintains stable row-wise order for edges with equal weights by default,
// because we collect them row-by-row from the upper triangular matrix.
int compareEdges(const void* a, const void* b) {
    Edge* e1 = (Edge*)a;
    Edge* e2 = (Edge*)b;
    if (e1->weight != e2->weight) {
        return e1->weight - e2->weight;
    }
    // If weights are equal, maintain the original row-wise order
    if (e1->u != e2->u) {
        return e1->u - e2->u;
```

```
    }
    return e1->v - e2->v;
}

void kruskalMST(int **cost, int V) {
    // Max possible edges in the upper triangular matrix of a graph with V ver
tices
    int maxEdges = (V * (V - 1)) / 2;
    Edge* edges = (Edge*)malloc(maxEdges * sizeof(Edge));
    int edgeCount = 0;

    // Collect all valid edges from the upper triangular matrix
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            // Check for valid edge (9999 represents infinity/no edge)
            if (cost[i][j] != 9999) {
                edges[edgeCount].u = i;
                edges[edgeCount].v = j;
                edges[edgeCount].weight = cost[i][j];
                edgeCount++;
            }
        }
    }

    // Sort edges by weight, keeping the row-wise upper triangular order for t
ies
    qsort(edges, edgeCount, sizeof(Edge), compareEdges);

    // Allocate memory for creating V subsets
    Subset* subsets = (Subset*)malloc(V * sizeof(Subset));
    for (int v = 0; v < V; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    int mstWeight = 0;
    int edgesInMST = 0;

    // Process sorted edges
    for (int i = 0; i < edgeCount && edgesInMST < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int set_u = find(subsets, u);
        int set_v = find(subsets, v);

        // If including this edge doesn't cause a cycle
        if (set_u != set_v) {
            printf("Edge %d:(%d, %d) cost:%d\n", edgesInMST, u, v, edges[i].we
ight);
            mstWeight += edges[i].weight;
            edgesInMST++;
            Union(subsets, set_u, set_v);
        }
    }
```

```
    // Print final minimum cost
    printf("Minimum cost= %d\n", mstWeight);

    // Clean up local allocations
    free(edges);
    free(subsets);
}


int main() {
    int V;
    printf("No of vertices: ");
    scanf("%d", &V);

    int **cost = (int **)malloc(V * sizeof(int *));
    for (int i = 0; i < V; i++)
        cost[i] = (int *)malloc(V * sizeof(int));

    printf("Adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &cost[i][j]);

    kruskalMST(cost, V);

    for (int i = 0; i < V; i++)
        free(cost[i]);
    free(cost);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
No of vertices:
5
Adjacency matrix:
9999 2 9999 9999 5
2 9999 3 9999 9999
9999 3 9999 4 9999
9999 9999 4 9999 9999
5 9999 9999 9999 9999
Edge 0:(0, 1) cost:2
Edge 1:(1, 2) cost:3
Edge 2:(2, 3) cost:4
Edge 3:(0, 4) cost:5
Minimum cost= 14

Test Case - 2
User Output

```
No of vertices:
4
Adjacency matrix:
9999 3 6 3
3 9999 5 2
6 5 9999 4
3 2 4 9999
Edge 0:(1, 3) cost:2
Edge 1:(0, 1) cost:3
Edge 2:(2, 3) cost:4
Minimum cost= 9
```